



Project Report

QubitFlow

Quantum Computing Project under the Programming Club, IIT Kanpur

August 2025

Project Repository

<https://github.com/pclubiitk/Qubit-Flow-25>

Contributors

*Aaryan Pruthi, Antriksh Rastogi, Ayush Yadav
Dhruvi Jain, Garvit Bhutani, Vinayak Srivastava
Mridul Garg, Shourya Mathur, Vaibhav Kalyan*

Mentors

*Advaith G S
Himanshu Sharma*

ABSTRACT

This report records the completion of the QubitFlow project under the Programming Club, IIT Kanpur. The goal of the project is to present an extensive yet simplistic journey into quantum computing by interweaving theoretical knowledge, programming proficiency, and experimental handling.

Starting with a background in basic concepts of quantum mechanics such as Dirac notation, states of qubits, tensor products, and entanglement, the report lays a good conceptual foundation. These are followed by practical work in using the Qiskit platform, where circuit building, manipulation, and visualization are discussed in detail. A central emphasis is placed on quantum primitives like samplers and estimators that support flexible and hardware-optimized execution. Classical and quantum complexity classes are contrasted to put the quantum computational power of algorithms in context. Simulation methods, both digital and variational, are subsequently presented, including the execution of the VQE algorithm and Heisenberg model simulations.

Transpilation is discussed extensively, as well as the latest advancements in transpiler strategies based on AI. The report continues to examine pillar algorithms of quantum computing: Deutsch-Jozsa, Grover's Search, and Shor's Algorithm. Particular focus is placed on modular exponentiation circuit, the place of QFT and QPE, and classical post-processing in Shor's approach. The last chapters present more advanced subjects like circuit cutting and quantum knitting, as well as quantum gradients via the parameter-shift rule and central difference approximation.

Overall, this report documents a comprehensive project that covers the conceptual, technical, and experimental aspects of quantum computing. It is presented both as a chronicle of learning and as a working base for further investigation in this new field.

CONTENTS

1	Introduction to Quantum Mechanics	4
1.1	Classical vs Quantum Paradigms	4
1.2	Dirac Notation (Bra-Ket Formalism)	4
1.3	Single Qubit States	5
1.4	Multi-Qubit Systems and Tensor Products	6
1.5	Quantum Entanglement	6
2	Quantum Computing with Qiskit	7
2.1	Introduction to Qiskit Framework	7
2.2	Quantum Circuit Development	8
2.3	Circuit Construction and Manipulation	8
3	Primitives in Quantum Computing	10
3.1	Primitives	10
3.2	Types of Qiskit Primitives	11
4	Quantum Simulations	12
4.1	Classical and Quantum Complexity	12
4.2	Quantum Simulations	12
4.3	Two Spin State Heisenberg Model	13
4.4	VQE	14
4.5	Steps Involved in VQE	14
5	Transpilers	16
5.1	Introduction to Transpilers	16
5.2	Stages of Quantum Circuit Transpilation	17
5.3	AI-Based Transpilers	19
6	Deutsch- Jozsa Algorithm	20
6.1	Introduction	20
6.2	The Quantum Algorithm	20
7	Grover's Algorithm	22
7.1	The Search Problem and its Classical Approach	22
7.2	Quantum Approach : Grover's Algorithm	22
7.3	Why $O(\sqrt{N})$? The Geometry of the Search	24
8	Shor's Algorithm	25
8.1	RSA and Why Factoring Matters	25
8.2	Algorithm Outline	25
8.3	Quantum Fourier Transform (QFT)	25
8.4	Quantum Phase Estimation (QPE)	26

8.5	Modular Exponentiation Circuit	27
8.6	Full Shor Circuit Structure	27
8.7	Conclusion	27
9	Quantum Knitting and Circuit Cutting	28
9.1	Introduction	28
9.2	Quantum Knitting	28
9.3	Motivation	28
9.4	Circuit Cutting	28
10	Quantum Gradients	30
10.1	Parameter- Shift Rule	30
10.2	Central Difference Approximation	31
10.3	Application to Variational Quantum Eigensolver	32

1 INTRODUCTION TO QUANTUM MECHANICS

1.1 Classical vs Quantum Paradigms

Classical physics provides a framework in which the behavior of physical systems can be predicted with arbitrary precision, given sufficient knowledge of initial conditions and forces. In this worldview, properties such as position and momentum have definite values at all times, and measurements merely reveal pre-existing values without altering the system. Quantum mechanics, on the other hand, reveals that at microscopic scales particles do not possess definite properties until they are measured. Instead, particles exist in superpositions of states, and the act of measurement itself plays an active role in determining the outcome.

Probabilistic Outcomes and Discrete States

Under classical mechanics, the laws of motion allow one to compute the exact trajectory of a particle if its initial position and velocity are known. By contrast, quantum mechanics replaces this deterministic evolution with a probabilistic description. The wavefunction provides amplitudes for different outcomes, and only the likelihood of each outcome can be predicted.

Classical observables such as the position of a particle can, in principle, vary continuously over a range of values. Quantum systems often exhibit quantized properties: only certain discrete energy levels or spin orientations are allowed. Furthermore, when a measurement is performed on a quantum system, the wavefunction collapses to a specific eigenstate of the measured observable.

1.2 Dirac Notation (Bra-Ket Formalism)

Dirac notation, also known as bra-ket notation, streamlines the algebra of quantum mechanics by using the symbols $|\psi\rangle$ (ket) and $\langle\phi|$ (bra) to represent state vectors and their duals, respectively.

- **Ket** $|\psi\rangle$: Represents a quantum state as a column vector in a complex Hilbert space.
- **Bra** $\langle\phi|$: Denotes the Hermitian conjugate (or complex conjugate transpose) of a ket, yielding a row vector.
- **Inner product** $\langle\phi|\psi\rangle$: Produces a complex scalar. The squared magnitude $|\langle\phi|\psi\rangle|^2$ gives the probability of finding the system in state $|\phi\rangle$ when it is prepared in $|\psi\rangle$.
- **Outer product** $|\psi\rangle\langle\phi|$: Defines a linear operator that projects any vector onto the state $|\psi\rangle$ along the direction of $\langle\phi|$.
- **Observables as Operators**: Physical observables correspond to Hermitian (self-adjoint) operators. The eigenvalues of these operators represent the possible outcomes of measurements.

1.3 Single Qubit States

The qubit is the fundamental unit of quantum information, analogous to the classical bit. Unlike a classical bit which can take a value of either 0 or 1, a qubit can exist in a coherent superposition of both basis states.

Mathematical Description of a Qubit

A general pure state of a qubit is written as:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

Here, α and β are called probability amplitudes, and their squared magnitudes, $|\alpha|^2$ and $|\beta|^2$, represent the probabilities of measuring the qubit in states $|0\rangle$ or $|1\rangle$, respectively. The global phase of a state has no physical significance, as states differing only by a overall phase factor produce identical measurement statistics.

Geometric Representation on the Bloch Sphere

Any pure qubit state can be visualized by two angles θ and ϕ on the surface of the Bloch sphere, a unit sphere centered at the origin in three-dimensional space:

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle, \quad 0 \leq \theta \leq \pi, \quad 0 \leq \phi < 2\pi.$$

In this representation, the north pole of the sphere corresponds to $|0\rangle$, the south pole to $|1\rangle$, and points on the equator to equal superpositions of these states. Rotations on the Bloch sphere correspond to unitary operations acting on the qubit.

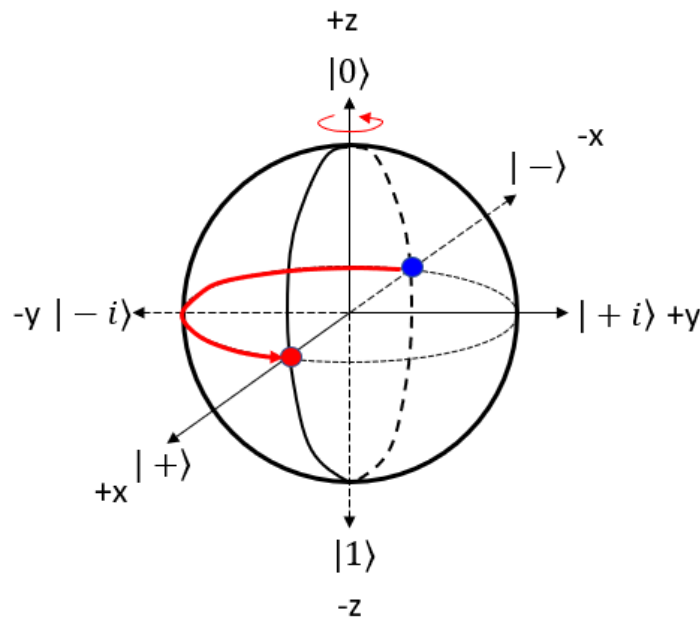


Figure 1: Visualization of a single-qubit state on the Bloch sphere, showing angles θ and ϕ .

1.4 Multi-Qubit Systems and Tensor Products

When multiple qubits are combined into a composite system, the state space grows exponentially via the tensor product of individual qubit spaces. This exponential growth underlies both the power and the complexity of quantum computing.

If two qubits are in states $|a\rangle$ and $|b\rangle$ respectively, the combined state of the two-qubit system is given by the tensor product:

$$|a\rangle \otimes |b\rangle = |ab\rangle.$$

As an explicit example, the tensor product of the basis states $|0\rangle$ and $|1\rangle$ is

$$|0\rangle \otimes |1\rangle = |01\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

Note that the dimension of the state vector doubles with each additional qubit.

1.5 Quantum Entanglement

A composite state $|\Psi\rangle_{AB}$ of qubits A and B is entangled if it cannot be factorized as a tensor product $|a\rangle_A \otimes |b\rangle_B$. Such states exhibit statistical correlations that are stronger than those allowed by classical physics.

Non-Local Correlations and Measurement

Entangled qubits remain correlated even when spatially separated by large distances. A measurement on one qubit immediately affects the state of the other, a phenomenon that was famously criticized by Einstein as “spooky action at a distance” and later confirmed experimentally through violations of Bell inequalities.

2 QUANTUM COMPUTING WITH QISKIT

2.1 Introduction to Qiskit Framework

Qiskit serves as the bridge between abstract quantum theory and practical implementation. By offering a layered architecture and user-friendly interface, it allows users to create, simulate, and run quantum programs on actual quantum hardware.

Whether you're a researcher developing algorithms or a student learning basic quantum mechanics, Qiskit provides the tools necessary to engage directly with quantum computing. Its modular design makes it so that every user can work with only those elements they require, from circuit construction and visualization to transpilation and back-end execution.

Definition and Origins

Qiskit (Quantum Information Software Kit) is an open-source, Python-based framework for quantum computing developed by *IBM Research* and released in **2017**. It was launched soon after the IBM Quantum Experience in 2016, which allowed public access to quantum computers via the cloud. This was a pivotal moment that enabled educational institutions, developers, and researchers to interact with real quantum hardware.

Core Philosophy and Architecture

Qiskit adopts a circuit-based quantum computation model. It provides high-level abstractions to define quantum logic, supports various simulators and backends, and encourages modular development. This allows users to build reusable components and adapt circuits for different hardware platforms.

Role in the Quantum Ecosystem

Qiskit's purpose goes far beyond tooling; it's an entire ecosystem. It is commonly used in academia and business for quantum algorithm prototyping, simulations, and real-device testing. Its rich documentation and open-source support enable teamwork and rapid development.

For full features and runtime integration:

Full Installation

```
pip install qiskit[all]
```

IBM Runtime Installation

```
pip install qiskit-ibm-runtime
```

Python-Based Design

Built on Python, Qiskit integrates seamlessly with the scientific ecosystem, enabling users to utilize packages like NumPy and Matplotlib for classical logic and visualization. This design simplifies rapid prototyping and hybrid quantum-classical workflows.

2.2 Quantum Circuit Development

Circuit development is the central task in quantum programming. Just like how classical programs are constructed based on logic gates and control flow, quantum programs are constructed based on quantum gates applied sequentially to qubits. Qiskit also offers a nice Python API to describe these operations, making it easy for users to encode algorithms such as Deutsch-Jozsa, Grover's, or Shor's in a few lines of code.

Single-Qubit Gates

These gates operate on individual qubits and are used to initialize, rotate, or superpose states. They form the building blocks for complex quantum logic.

Single-Qubit Gates

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(1)
qc.x(0)
qc.h(0)
```

The X gate flips the qubit, and the H gate places it in superposition.

Multi-Qubit Gates

Multi-qubit gates enable interactions between qubits and are crucial for creating entanglement. These gates are the heart of quantum parallelism, allowing conditional operations that underpin nearly all quantum algorithms.

Multi-Qubit Gates

```
qc = QuantumCircuit(2)
qc.cx(0, 1)
qc.cz(0, 1)
```

CNOT and CZ gates perform control-dependent actions, forming the backbone of quantum algorithms.

2.3 Circuit Construction and Manipulation

Aside from specifying which gates to use, quantum development is concerned with controlling how circuits are organized and run. This entails giving registers names, building circuits modularly, and

seeing operations for checking. Qiskit facilitates complete control over these things, so it's applicable to simple prototypes as well as intricate algorithmic pipelines.

QuantumCircuit Class Fundamentals

This class enables low-level creation and management of quantum and classical registers. Its flexibility allows naming, tracking, and measuring multiple qubits.

Creating Registers

```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit

qr = QuantumRegister(2)
cr = ClassicalRegister(2)
qc = QuantumCircuit(qr, cr)
```

This code sets up a 2-qubit circuit ready for manipulation and measurement.

Circuit Visualization Techniques

Qiskit supports graphical, ASCII, and LaTeX-style circuit visualization for better interpretation.

Visualizing Circuit

```
qc.measure_all()
qc.draw(output='mpl')
```

The Matplotlib output gives a polished view of gates and measurement.

Circuit Composition

Modular circuit construction increases the reusability and readability of quantum programs. Rather than reconstructing everything in one place, programmers can design and implement individual blocks of logic and then plug them together into a finished circuit. This process is parallel to software engineering techniques and is particularly valuable when constructing large-scale quantum algorithms that involve iterative or nested structures.

Composing Circuits

```
qc1 = QuantumCircuit(2)
qc1.h(0)
qc1.cx(0, 1)

qc2 = QuantumCircuit(2)
qc2.x(0)

qc3 = qc1.compose(qc2)
```


Estimator Usage Example

```
estimator = EstimatorV2(mode=backend)
estimator_job = estimator.run([
    (circuit1, observable1, param_values1),
    (circuit2, observable2, param_values2),
])
```

Sampler

The Sampler's core task is sampling the output register from the execution of one or more quantum circuits. The input circuits can be parametrized, as long as the parameter values are also provided as input to the primitive.

The input is one or more PUBs, in the format:

(single circuit, one or more optional parameter value, optional shots),

where there can be multiple parameter values items, and each item can be either an array or a single parameter, depending on the chosen circuit. Additionally, the input must contain measurements. The output is counts or per-shot measurements, as PubResult objects, without weights.

Sampler Usage Example

```
sampler = SamplerV2(mode=backend)
sampler_job = sampler.run([
    (circuit1, param_values1, shots1),
    (circuit2, param_values2, shots2),
])
```

3.2 Types of Qiskit Primitives

There are two types of Qiskit primitives: the base classes, and their implementations. The Qiskit primitives are defined by open-source primitive base classes that live in the Qiskit SDK (in the `qiskit.primitives` module). Providers (such as Qiskit Runtime) can use these base classes to derive their own Sampler and Estimator implementations. Most users will interact with provider implementations, not the base primitives

The `qiskit.primitives` module defines two abstract base classes, `BaseEstimatorV2` and `BaseSamplerV2`, which provide a standard interface for building primitives-based execution models. Implementations include: Qiskit Runtime primitives (`EstimatorV2`, `SamplerV2`) for cloud-based IBM Quantum hardware with advanced features such as error mitigation; `StatevectorEstimator` and `StatevectorSampler` for ideal statevector simulations using Qiskit's built-in simulator; and `BackendEstimatorV2` and `BackendSamplerV2`, which wrap any quantum backend into a primitive interface, allowing primitive-style code for providers without native support.

4 QUANTUM SIMULATIONS

4.1 Classical and Quantum Complexity

Before exploring how quantum systems are simulated on real or virtual hardware, it is essential to understand the theoretical framework that defines the potential and limits of quantum computation. Classical complexity theory categorizes problems based on the time required for a classical computer to solve them, with two key classes being P (problems solvable in polynomial time, e.g., searching, sorting, arithmetic) and NP (problems whose solutions can be verified in polynomial time, e.g., solving a Rubik's cube or Sudoku). Since any problem in P can also be verified in polynomial time, $P \subseteq NP$. Within NP, NP-Hard problems are those to which every NP problem can be reduced in polynomial time (not necessarily in NP), while NP-Complete problems are both in NP and as hard as any NP problem, meaning all NP problems reduce to them in polynomial time.

In the quantum setting, the class BQP (Bounded-error Quantum Polynomial time) contains problems solvable on a quantum computer in polynomial time with bounded error. A famous example is Shor's algorithm for integer factorization. Since quantum computers can also execute classical algorithms, $P \subseteq BQP$. Quantum computing challenges classical complexity assumptions by offering potential speedups for certain problems once thought to be intractable, bridging classical and quantum paradigms in computational theory.

4.2 Quantum Simulations

Quantum simulations refer to the use of quantum computing to tackle tasks that are highly complex for classical computers in terms of time and space complexity. There are two broad types of these simulations:

Digital Quantum Simulation (DQS)

These kinds of simulations involve creating a particular model using Qiskit, quantum gates, and quantum circuits.

For example: simulating a 2-spin state Heisenberg model on Qiskit and observing the time evolution of the state vector.

Analog Quantum Simulation (AQS)

AQS refers to simulations where a less accessible quantum problem is mapped to another quantum problem with controllable dynamics. These simulations do not involve Qiskit or quantum gates, and instead use analog simulators. For example: the model of trapped ions maps the Ising model.

4.3 Two Spin State Heisenberg Model

The model refers to the interaction between two spin-1/2 particles. The Hamiltonian of the system is:

$$H = \frac{J}{4}(\sigma_1^x \sigma_2^x + \sigma_1^y \sigma_2^y + \sigma_1^z \sigma_2^z), \quad J : \text{exchange interaction constant}$$

σ^x : Pauli X gate σ^y : Pauli Y gate σ^z : Pauli Z gate

There are four steps involved in modeling the time evolution of this 2-qubit system on Qiskit:

1. **Making the Quantum Circuit:** We have to initialize our 2 input qubits and apply RXX, RYY, and RZZ gates, which are time-evolved exponentials of XX, YY, and ZZ gates.

Code snippet to implement the circuit

```
qc = QuantumCircuit(2)
t = Parameter("Time")
qc.x(0)
qc.append(RXXGate(2 * t), [0, 1])
qc.append(RYYGate(2 * t), [0, 1])
qc.append(RZZGate(2 * t), [0, 1])
qc.measure_all()
```

2. **Importing the Backend:** The backend can be a local simulator like AerSimulator or a real-time backend from IBM Quantum.
3. **Transpiling and Running on the Backend:** The circuit is transpiled and executed using a primitive like the sampler, which returns shot-by-shot bit strings.

Running the circuit on backend

```
qc_transpiled = transpile(qc, backend)
sampler = Sampler(backend=backend)
sampler.run([qc_transpiled], shots=1024)
```

4. **Post-processing and Plotting:** The output from the sampler is in bit string form. To extract meaningful results, we convert the bit counts to probabilities and visualize them using graphs.

The attached notebook simulates the time evolution of the state vector using the 2-spin state Heisenberg model: [Two spin state Heisenberg Model Link](#)

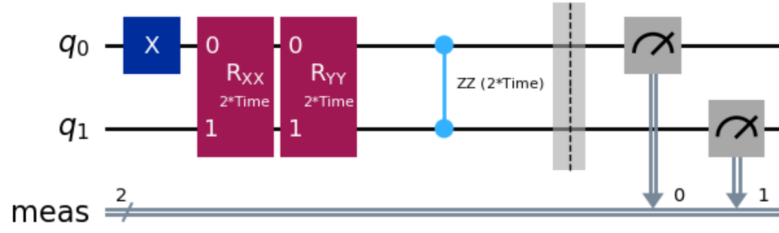


Figure 3: Circuit Diagram for the 2-spin state model

4.4 VQE

VQE or Variational Quantum Eigensolver refers to algorithms used to find the ground state of a system by iteratively improving upon a parameterized quantum circuit called an "ansatz". Classical methods for this task scale poorly with system size, making them infeasible for large problems. This algorithm finds high relevance in various fields of quantum chemistry. In the given example, we shall try to find the ground state of a system by minimizing the expectation value of the Hamiltonian operator with respect to the eigenstate.

The example consists of three qubits with their Hamiltonian denoted as:

$$H = Z_0 + Z_1 + Z_2 \quad (1)$$

where Z_i = Pauli Z Gate on i -th qubit

Cost function is:

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (2)$$

4.5 Steps Involved in VQE

1. **Making the Circuit:** As defined earlier, the circuit consists of 3 qubits. Parameterized R_y and R_z gates are applied on each qubit, followed by entanglement via CNOT gates.

The VQE circuit

```
num_qubits = 3
rotation_blocks = ["ry", "rz"]
entanglement_blocks = "cz"
entanglement = "full"
ansatz = TwoLocal(num_qubits, rotation_blocks,
                  entanglement_blocks, entanglement, reps=1,
                  insert_barriers=True)
```

2. Transpilation and Hamiltonian Definition:

This step involves two sequential stages that prepare the quantum circuit and define the system's Hamiltonian:

Transpilation and Hamiltonian Definition

```
from qiskit.transpiler.preset_passmanagers import
generate_preset_pass_manager
from qiskit_ibm_runtime.fake_provider import FakeSherbrooke

backend_answer = FakeSherbrooke()
optimization_level_answer = 3
pm = generate_preset_pass_manager(
    backend=backend_answer,
    optimization_level=optimization_level_answer
)
isa_circuit = pm.run(ansatz)

# defining the hamiltonian
from qiskit.quantum_info import SparsePauliOp
pauli_op = SparsePauliOp(['ZII', 'IZI', 'IIZ'])
hamiltonian_isa = pauli_op.apply_layout(layout=isa_circuit.layout)
```

The attached notebook simulates the VQE algorithm: VQE Simulator

5 TRANSPILERS

5.1 Introduction to Transpilers

In classical computing, a transpiler (short for translating compiler) is a tool that converts source code from one high-level language into another. Unlike traditional compilers that convert source code into low-level machine instructions or bytecode, transpilers perform a source-to-source transformation. You can use a transpiler to convert a program from TypeScript to JavaScript, while fully preserving its original logic and behavior.

The idea of transpilation is central to quantum computing due to the inherent nature of quantum hardware. Unlike classical processors that can execute abstract instructions relatively freely, quantum hardware has deep architectural constraints:

- Only specific types of quantum gates are natively supported.
- Not all qubits are physically connected.
- Each device has its own layout, coherence times, and error rates.

A quantum transpiler, such as the one provided by IBM's Qiskit SDK, is responsible for this essential transformation. When you call `transpile(circuit, backend)`, the Qiskit transpiler:

- Maps logical qubits to physical qubits.
- Inserts SWAP gates where direct connectivity is missing.
- Converts abstract gates into native basis gates of the backend.
- Optimizes the circuit to reduce total depth and gate error.

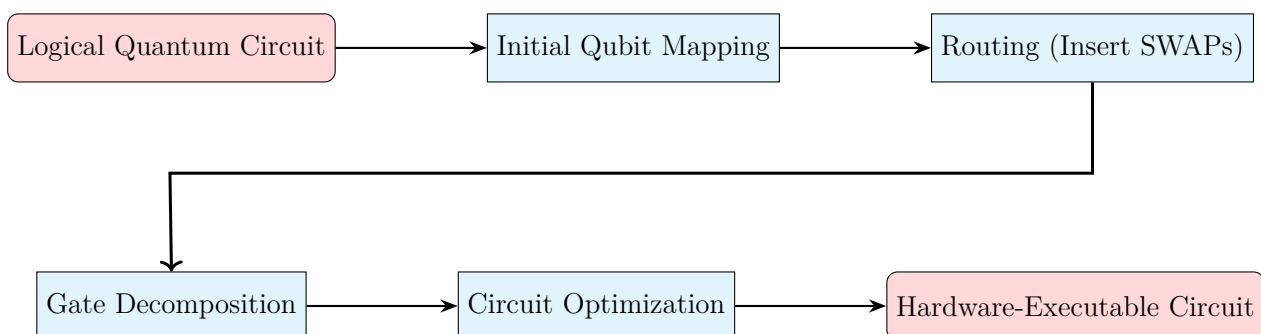


Figure 4: The Quantum Transpilation Pipeline

Qiskit's transpiler is built to be modular and extensible. It is composed of many small transformation units called passes. These passes are managed and orchestrated using:

- **PassManager**, a container of passes that apply sequential transformations.
- **StagedPassManager**, a high-level controller that breaks transpilation into logical stages and applies appropriate passes at each stage.

5.2 Stages of Quantum Circuit Transpilation

The process of quantum circuit transpilation typically involves a sequence of systematic stages. Each stage transforms the original logical circuit to make it compatible with specific quantum hardware, while optimizing for circuit efficiency, execution time, and hardware constraints.

Initialization Stage

The Initialization (or the *init* stage) is probably the simplest stage of the transpilation pipeline. In this stage, Qiskit constructs a standardised internal data structure that represents the circuit in terms of its qubits, classical bits, gates and their dependencies. This helps the subsequent passes run smoothly by providing them with a unified version of the circuit.

Also, Qiskit initialises various data tracking mechanisms to store the metadata of the circuit, which is accessed by different passes during transpilation. Below is a representative example of such metadata (often stored in the `property_set`):

Sample Circuit Properties

```
{
  'depth': 12, 'width': 5, 'size': 34,
  'num_qubits': 5, 'num_clbits': 5, 'operations': {
    'cx': 12, 'h': 5, 'rz': 7, 'measure': 10
  },
  'is_swap_mapped': False, 'layout': None, 'initial_layout': None,
  'condition_bits': set(), 'qubit_connectivity': 'undirected'
}
```

These mechanisms act as a memory for the transpiler, allowing later passes to make decisions based on circuit properties that have already been analyzed or inferred. Sometimes, an early unrolling of complex gates into simpler basis is also performed in this step to ease out burden on the other stages.

Layout Stage

In this stage, a mapping is established between the logical qubits of the circuit and the physical qubits of the target quantum device. Since quantum hardware imposes physical connectivity constraints, i.e., not all qubit pairs can interact directly, this mapping has a significant impact on the efficiency of the final circuit.

Qiskit tries to allocate logical qubits to physical qubits such that the need for using additional routing operations like SWAP gates is minimized. Several strategies can be used here, such as trivial mapping (mapping the logical qubit q_i to physical qubit q_i), dense mapping (prioritizing closely connected physical qubits), or noise-adaptive mapping (accounting for fidelity and error rates on physical qubits).

Routing Stage

In quantum devices, not all qubits are physically connected to each other, and two-qubit gates (such as CNOT) can only be applied between adjacent qubits in the hardware coupling map. When a circuit includes operations between qubits that are not directly connected on the physical device, it becomes necessary to bring them into proximity. This is achieved by inserting SWAP gates to move qubit states along the connectivity graph until the required qubits are adjacent. Minimizing the use of SWAP gates is essential for effective circuit execution (due to their high overhead fidelity).

Translation Stage

The Translation stage focuses on converting abstract, high-level quantum gates into a sequence of basis gates supported by the target backend. Each quantum device has a native gate set, often limited to a small number of elementary operations. For instance, a hardware backend may only support gates like u_1 , u_2 , u_3 , and CNOT. Any operation outside this set must be decomposed into an equivalent sequence using only the supported gates.

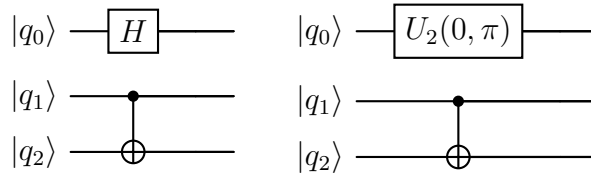


Figure 5: Translating high-level gates into basis gates. The Hadamard gate is decomposed into a $U_2(0, \pi)$ gate as supported by the backend.

Optimization Stage

The optimization stage's goal is to reduce the depth, gate count, and overall error accumulation of the circuit for execution on Noisy Intermediate-Scale Quantum (NISQ) devices. Qiskit applies multiple types of optimization passes (individual transformations), including Gate Cancellation, Gate merging and Commutation Analysis during this process.

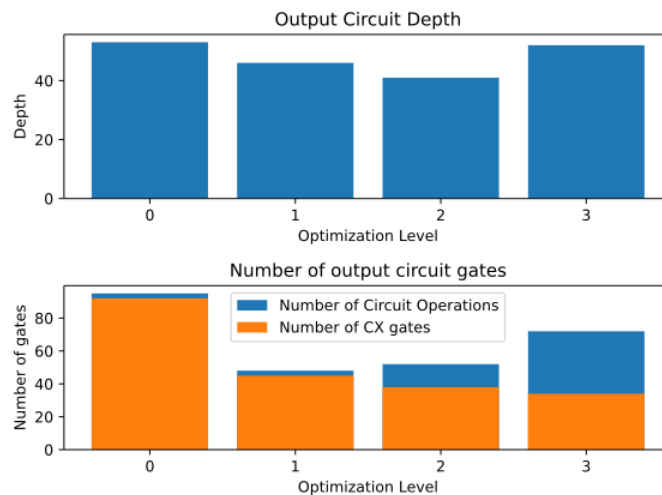


Figure 6: Effect of transpiler optimization level on (a) circuit depth and (b) number of gates including CNOTs.

Scheduling Stage

Scheduling, which only runs if explicitly invoked (similar to the *init* stage), involves assigning explicit timing to the operations in a quantum circuit based on the backend's gate durations and timing constraints. This process introduces deliberate time delays and accounts for idle periods between operations.

Qiskit provides support for multiple scheduling strategies. Commonly used ones include **as-soon-as-possible** (ASAP) and **as-late-as-possible** (ALAP), which respectively schedule gates as early or as late as dependencies allow. The appropriate method can be selected depending on the experimental requirements, hardware constraints, or optimization goals.

Transpilation with ALAP Scheduling

```
transpiler = transpile(qc,  
    backend=backend,  
    optimization_level=4,  
    scheduling_method='alap')
```

5.3 AI-Based Transpilers

Traditional transpilers rely heavily on rule-based heuristics, which are efficient and interpretable but often limited in adaptability and expressiveness. An AI-based transpiler, on the other hand, employs techniques from artificial intelligence, such as machine learning (ML), reinforcement learning (RL), and graph theory, to learn, predict, or evolve optimal transpilation strategies. Unlike rule-based systems that apply fixed patterns or hardcoded heuristics, AI-based transpilers are data-driven and capable of adapting to varying hardware conditions, circuit topologies, and performance requirements.

The core idea is to formulate transpilation as an optimization problem. The objective is to discover a transformation pipeline that minimizes a cost function, which may represent total error, circuit depth, SWAP count, or total execution time. Artificial intelligence techniques provide a more flexible and scalable approach to exploring this complex optimization space compared to handcrafted rules.

One major advantage of AI-based transpilers is their ability to dynamically adapt to changing or noisy hardware conditions, such as fluctuating qubit fidelities or decoherence rates. Moreover, they can generalize across diverse classes of quantum circuits and algorithms, making them versatile tools for real-world applications.

As quantum hardware continues to evolve, intelligent transpilation strategies will play a central role in optimizing the performance of quantum algorithms. AI-based transpilers mark an exciting and forward-looking development in this landscape.

6 DEUTSCH- JOZSA ALGORITHM

6.1 Introduction

Deutsch's algorithm is a quantum algorithm designed to determine whether a given Boolean function is constant or balanced. (A balanced boolean function is a function that outputs 1 for exactly half of the possible inputs and 0 for the other half, while a constant boolean functions is a function that outputs the same value either 0 or 1 for all of the possible inputs)

Classically, determining whether a function is constant or balanced may require evaluating the function on up to $2^{n-1} + 1$ inputs in the worst case (for an n -bit input). However, using quantum computing, the Deutsch–Jozsa algorithm can solve this problem with a single query to the function, demonstrating exponential speed-up.

6.2 The Quantum Algorithm

To leverage the power of quantum computation, we need to make all operations reversible. This includes the function evaluation, which we achieve by adding an auxiliary qubit and defining a unitary oracle as follows:

$$U_f : |x\rangle |y\rangle \mapsto |x\rangle |y \oplus f(x)\rangle$$

If we initialize the second qubit to $|0\rangle$, the oracle outputs:

$$U_f : |x\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle$$

This definition naturally generalizes to functions on n -bit inputs.

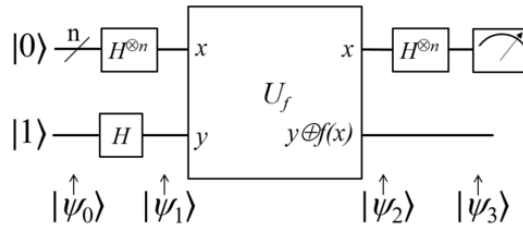


Figure 7: Circuit diagram for the Deutsch–Jozsa algorithm

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |1\rangle$$

Apply Hadamard gates to all qubits:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad H |1\rangle = |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Thus, the state becomes:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |-\rangle$$

Apply the oracle U_f :

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle |-\rangle$$

Since $|-\rangle$ is unaffected by further gates, we omit it for simplicity in further analysis.

Apply Hadamard gates again to the first n qubits:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)+x \cdot z} |z\rangle$$

We now focus on the amplitude of the $|0\rangle^{\otimes n}$ state, which is:

$$\frac{1}{2^n} \sum_x (-1)^{f(x)}$$

We consider two cases:

1. If f is constant:

- If $f(x) = 0$ for all x , then $(-1)^{f(x)} = 1$, and the sum becomes 2^n , resulting in amplitude 1.
- If $f(x) = 1$ for all x , then $(-1)^{f(x)} = -1$, and the sum becomes -2^n , resulting in amplitude -1 .

In both subcases, the square of the amplitude is 1, so measurement yields $|0\rangle^{\otimes n}$ with certainty.

2. If f is balanced:

Half the values of $f(x)$ are 0 and half are 1, so exactly half of the terms in the sum are $+1$ and half are -1 , leading to total sum zero, and hence, amplitude zero for $|0\rangle^{\otimes n}$.

Therefore, on measurement, we will never observe $|0\rangle^{\otimes n}$.

Conclusion: Thus, we can determine with certainty whether the function f is constant or balanced in just one query, i.e. one step.

7 GROVER'S ALGORITHM

7.1 The Search Problem and its Classical Approach

This is a classical problem in computer science. The problem refers to searching a particular element from a unstructured array of elements. The Grover's Algorithm deals with the same problem but with the quantum approach.

The classical approach is to check each and every element in the unstructured list. This time complexity for this approach is of the order of $O(n)$. So for very large value of n , this approach is very time consuming.

7.2 Quantum Approach : Grover's Algorithm

In 1996, Lov Grover, while at Bell Labs, proposed a quantum algorithm that provides a significant speedup for unstructured search problems. This algorithm provided a quadratic speedup to the Search problem. The time complexity for the algorithm was of the order of $O(\sqrt{N})$. This speedup was substantial for large databases.

The Algorithm

The heart of the algorithm lies in a technique called Selective Amplitude Amplification. It doesn't "search" in the classical sense; instead, it systematically evolves a quantum state so that upon measurement, the result is the desired marked element for which the search was implemented, with very high probability.

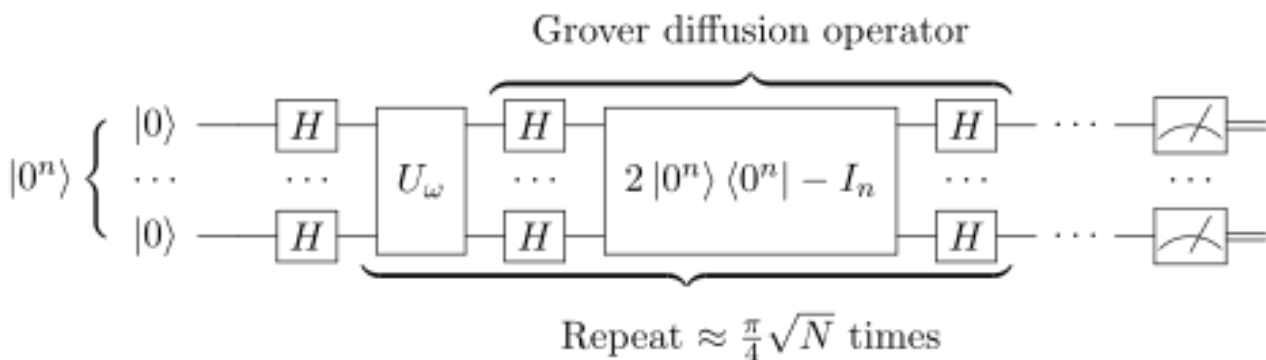


Figure 8: Circuit Diagram for Grover's Algorithm Implementation

Step 1: The Initialization

The algorithm begins with an n -qubit register, where $N = 2^n$, is the size of our search space. We start in the ground state $|0\dots 0\rangle$ and apply a Hadamard gate (H) to each qubit. This creates a uniform

superposition, where every possible state $|x\rangle$ has the same amplitude.

$$|s\rangle = H^{\otimes n} |0\dots 0\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

At this point, measuring the state gives a completely random result, as all outcomes are equally likely.

Step 2: The Oracle (Phase Inversion)

The "oracle" is a black-box function that can recognize the marked item, $|w\rangle$, without telling us which one it is. It does this by flipping the sign (the phase) of the marked state's amplitude.

$$U_w |x\rangle = (-1)^{f(x)} |x\rangle, \quad \text{where } f(x) = \begin{cases} 1 & \text{if } x = w \\ 0 & \text{otherwise} \end{cases}$$

This single operation is the only part of the algorithm that depends on the item we are searching for. After applying the oracle, the amplitude of $|w\rangle$ is negative, while all others remain unchanged.

Phase Inversion Oracle

```
def oracle_3():
    qc = QuantumCircuit(3)
    qc.x(1)
    qc.h(2)
    qc.ccx(0,1,2)
    qc.h(2)
    qc.x(1)
    return qc
```

Above is the code for creating our custom black box or the oracle function which takes into consideration that the marked state is $|101\rangle$.

Step 3: The Diffusion Operator (Inversion About the Mean)

This is the core of amplitude amplification. The diffusion operator, U_s , selectively amplifies the amplitude of the marked state. It works by reflecting every amplitude about the average amplitude. Since the marked state's amplitude is now negative and all others are positive, the average amplitude is very close to zero. Reflecting about this average dramatically increases the marked state's amplitude while slightly decreasing all others. The code for inversion about the mean amplitudes of a general n-qubit state is as follows:

Diffuser Circuit

```
def diffuser(nqubits):
    qc = QuantumCircuit(nqubits)
    qc.h(range(nqubits))
    qc.x(range(nqubits))
    qc.h(nqubits-1)

    qc.mcx(list(range(nqubits - 1)), nqubits - 1) # multi-controlled-toffoli

    qc.h(nqubits-1)
    qc.x(range(nqubits))
    qc.h(range(nqubits))
    return qc
```

The above code inverts all the states with respect to the mean but the catch here is that the our marked state which is already having negative amplitude ,its amplitude gets positive and doubled. But the states having positive amplitude, those states' amplitude get reduced.

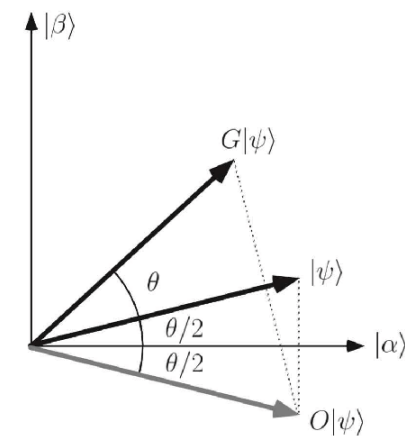


Figure 9: Inversion about mean

The steps 2 and 3 have to be repeated $\sqrt{N}$ times in order to reach the desired marked state.

7.3 Why $O(\sqrt{N})$? The Geometry of the Search

The geometric picture explains why the algorithm is so efficient. It all comes down to the angle of rotation. Here $|s\rangle$ is the initial state and $|w\rangle$ is the desired state.

In Grover's algorithm, the initial state $|s\rangle$ has amplitude $\frac{1}{\sqrt{N}}$ along the target state $|w\rangle$, giving $\sin(\theta) = \frac{1}{\sqrt{N}}$. For large N , $\theta \approx \frac{1}{\sqrt{N}}$ by the small-angle approximation. Each Grover iteration (oracle + diffusion) rotates the state vector by 2θ , and to reach the target at $\frac{\pi}{2}$, we require $R \times 2\theta \approx \frac{\pi}{2}$, hence $R \approx \frac{\pi}{4\theta} \approx \frac{\pi}{4}\sqrt{N}$. This yields the well-known $O(\sqrt{N})$ complexity, a quadratic speedup over the classical $O(N)$ search.

8 SHOR'S ALGORITHM

Let N be a large composite number (typically a product of two distinct primes, p and q). The goal is to find a non-trivial factor of N efficiently.

Classical factoring algorithms, such as the General Number Field Sieve, scale sub-exponentially, but not polynomially. Shor's algorithm achieves factoring in polynomial time on a quantum computer by reducing the problem to period-finding.

8.1 RSA and Why Factoring Matters

RSA encryption relies on the fact that given $N = pq$, it is computationally infeasible to find p and q given only N . The public key contains N and an exponent e , while the private key requires knowledge of the totient $\phi(N) = (p-1)(q-1)$, which is easy to compute if p and q are known. Hence, factoring N breaks RSA.

8.2 Algorithm Outline

Given an integer N and a randomly chosen integer a such that $\gcd(a, N) = 1$, the goal is to find the period r of the function:

$$f(x) = a^x \mod N$$

Using Quantum Phase Estimation (QPE) and Quantum Fourier Transform (QFT), we can determine the period r efficiently.

8.3 Quantum Fourier Transform (QFT)

The Quantum Fourier Transform is the quantum analog of the classical Discrete Fourier Transform (DFT). It maps a computational basis state $|x\rangle$ to a quantum superposition with amplitudes given by complex roots of unity. Mathematically, for an n -qubit system:

$$\text{QFT}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k / 2^n} |k\rangle$$

This transformation plays a crucial role in many quantum algorithms, especially those involving periodicity, such as Shor's algorithm.

The QFT circuit consists of Hadamard gates followed by controlled phase rotations, and finally swap gates to reverse the order of qubits.

Quantum Fourier Transform Circuit

```
from qiskit import QuantumCircuit
import numpy as np

def qft(n):
    qc = QuantumCircuit(n)
    for j in range(n):
        qc.h(j)
        for k in range(j+1, n):
            qc.cp(np.pi / 2**(k-j), k, j)
    for i in range(n//2):
        qc.swap(i, n - i - 1)
    return qc
```

8.4 Quantum Phase Estimation (QPE)

Quantum Phase Estimation is an algorithm that estimates the phase ϕ in the eigenvalue of a unitary operator U when, $U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle$.

Here, $|\psi\rangle$ is an eigenstate of U , and $0 \leq \phi < 1$. The phase ϕ is encoded in a register of qubits using the inverse QFT. The QPE algorithm uses two quantum registers, a control register of t qubits initialized in a superposition (via Hadamards) and a target register in state $|\psi\rangle$.

The procedure involves first applying Hadamard gates to the control register, followed by controlled- U^{2^j} operations for each control qubit. Next, the inverse Quantum Fourier Transform (QFT) is applied to the control register, and finally, the control register is measured to estimate ϕ .

Quantum Phase Estimation Circuit

```
from qiskit.circuit.library import QFT
from qiskit import QuantumCircuit

def qpe(U, n_count):
    n = U.num_qubits
    qc = QuantumCircuit(n_count + n, n_count)
    qc.h(range(n_count))

    for i in range(n_count):
        qc.append(U.power(2**i).control(), [i] + list(range(n_count, n_count + n)))

    qc.append(QFT(num_qubits=n_count, inverse=True), range(n_count))
    qc.measure(range(n_count), range(n_count))
    return qc
```

8.5 Modular Exponentiation Circuit

This is the most resource-intensive part and must be implemented using controlled multipliers and adders modulo N . In practice, it's often abstracted as a unitary oracle.

Controlled Modular Exponentiation (Simplified)

```
def modexp_gate(a, N, power):
    # This is a placeholder for a unitary gate that performs:
    #  $|x\rangle \rightarrow |a^x \bmod N\rangle$ 
    pass
```

8.6 Full Shor Circuit Structure

The quantum portion of Shor's algorithm is designed to find the period r of the function $f(x) = a^x \bmod N$, which is essential for factoring the integer N . The full structure of the algorithm can be broken down into the following key steps:

1. **Initialization & Superposition:** Prepare a counting register of t qubits in $|0\rangle^{\otimes t}$ and a target register (for values mod N) in $|1\rangle$. Apply Hadamard gates to the counting register to create:

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle$$

2. **Controlled Modular Exponentiation:** Apply U_a so that:

$$U_a |x\rangle |1\rangle = |x\rangle |a^x \bmod N\rangle$$

controlled on each bit of x , encoding the periodicity of $a^x \bmod N$ into the counting register.

3. **Period Extraction:** Perform the inverse QFT on the counting register and measure it to obtain c , giving an estimate of k/r .
4. **Classical Processing & Factorization:** Use the continued fraction algorithm to recover r from $c/2^t$, verify $a^r \equiv 1 \bmod N$, and if r is even with $a^{r/2} \not\equiv -1 \bmod N$, compute:

$$\gcd(a^{r/2} \pm 1, N)$$

to find non-trivial factors of N .

8.7 Conclusion

Shor's algorithm demonstrates how quantum computing can solve specific problems exponentially faster than classical computers. Its successful execution on sufficiently large N would render RSA and similar cryptosystems insecure, highlighting the urgency for post-quantum cryptographic research.

9 QUANTUM KNITTING AND CIRCUIT CUTTING

9.1 Introduction

Quantum computing promises exponential speedups for various computational problems. However, Noisy Intermediate-Scale Quantum (NISQ) devices are currently limited by qubit count, decoherence, and gate errors. Quantum Knitting and Circuit Cutting aim to address these limitations by enabling large quantum algorithms to run on constrained hardware.

9.2 Quantum Knitting

Quantum Knitting refers to a class of methodologies designed to decompose large quantum circuits into smaller, more manageable subcircuits (or "fragments"), which can then be executed independently on smaller quantum devices or simulators. The results from these independent executions are later recombined using classical post-processing techniques to reconstruct the overall computation.

9.3 Motivation

These techniques are particularly valuable for running algorithms such as Shor's algorithm, quantum phase estimation, or quantum chemistry simulations on limited hardware. It is advantageous in ways including:

- Allowing Circuit decomposition and execution on smaller devices
- Conducting Parallel fragment execution
- Realising Hybrid quantum-classical computation workflows

9.4 Circuit Cutting

Suppose we have a quantum circuit that operates on qubits. We wish to cut this circuit along a particular qubit wire, effectively splitting the circuit into two subcircuits C1 and C2. The key idea is to express the identity operation at the cut point using a complete set of basis states: For a qubit wire q_i , we insert:

$$\rho = \sum_{x \in \{0,1\}} |x\rangle\langle x| \rho |x\rangle\langle x|$$

Each circuit fragment is evaluated with respect to input x and post-processed to reconstruct the overall circuit output.

Implementation Steps

1. Circuit Definition: Define the full quantum circuit that needs to be evaluated.

2. Identify Cut Points: Choose qubit wires where the circuit will be partitioned. These are typically between loosely entangled components.
3. Fragment Generation: Generate the subcircuits (fragments) by cutting at the chosen points and replacing the cut wires with classical control and measurement gates.
4. Fragment Execution: Simulate or run each fragment separately on available quantum backends.
5. Result Reconstruction: Combine the measurement results from all fragments using classical computation to reconstruct the original circuit's behavior.

Circuit Cutting Example

```
from qiskit.circuit import QuantumCircuit
from qiskit.circuitcutting import cut_circuit, reconstruct_expectation_values

qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)

fragments, submaps = cut_circuit(qc, cuts=[(1, 'out'), (1, 'in')])
# Execute fragments...
# Reconstruct result
# final_result = reconstruct_expectation_values(...)
```

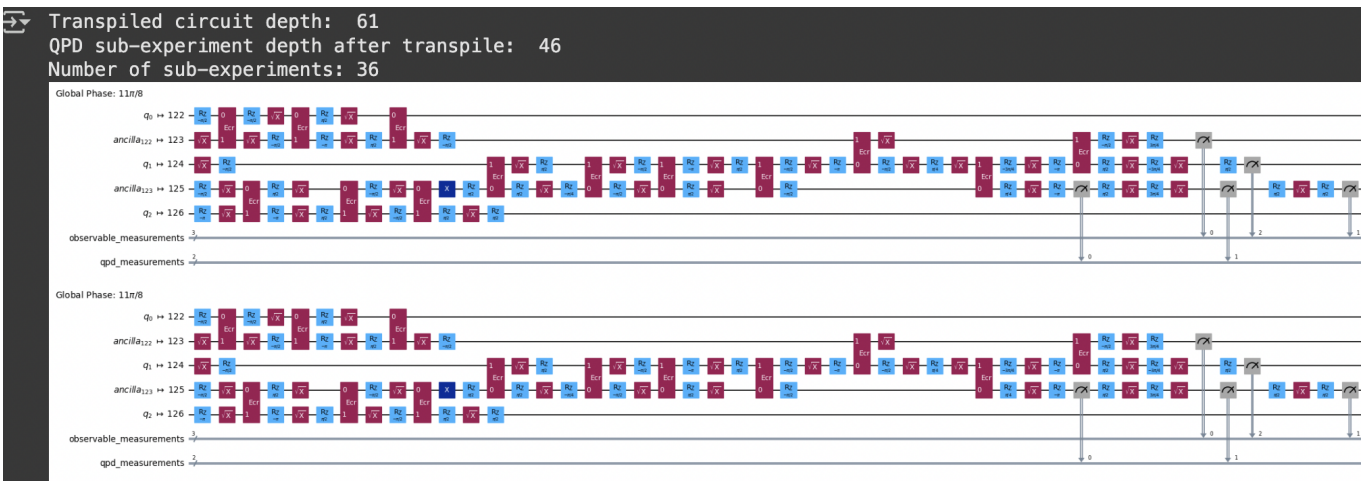


Figure 10: Circuit depth before and after knitting

10 QUANTUM GRADIENTS

A popular problem in quantum computing is that of finding the ground state, or more generally, the state with the lowest energy of a given Hamiltonian. This is often approached by preparing a parameterized quantum state and optimizing its energy expectation value. To do this efficiently, it is necessary to understand how the expectation value of a quantum observable changes as the circuit parameters are varied. This is where quantum gradients are introduced.

10.1 Parameter- Shift Rule

A variational quantum circuit acting on n qubits is a sequence of gates depending on a set of classical parameters $\theta = (\theta_1, \theta_2, \dots, \theta_m)$ and producing a family of unitary operations $U(\theta)$. Typically, the circuit is applied to a fixed reference state $|0\rangle$ and the expectation value of a Hermitian observable M is measured:

$$f(\theta) = \langle 0 | U(\theta)^\dagger M U(\theta) | 0 \rangle.$$

Considering this expectation value as the cost function itself, the required gradient (of an arbitrary order d) becomes:

$$g_{j_1 j_2 \dots j_d}(\theta) = \frac{\partial^d f(\theta)}{\partial \theta_{j_1} \partial \theta_{j_2} \dots \partial \theta_{j_d}}.$$

A typical structure of many variational circuits which can be executed by near-term quantum computers is:

$$U(\theta) = V_m U_m(\theta_m) \dots V_2 U_2(\theta_2) V_1 U_1(\theta_1),$$

where V_j is a constant arbitrary circuit, while $U_j(\theta_j)$ is a rotation-like gate, i.e., characterized by a generator H_j such that $H_j^2 = I$ and thus,

$$U_j(\theta_j) = e^{-(i/2)H_j\theta_j} = \cos(\theta_j/2)I - i \sin(\theta_j/2)H_j$$

First- order derivative

The following is a very simple yet powerful trigonometric identity with respect to gradients:

$$\frac{dT}{dx} = \frac{T(x+h) - T(x-h)}{2 \sin(h)},$$

where T refers to any of the trigonometric functions $\sin$ and $\cos$ (and $h \neq k\pi, k \in \mathbb{Z}$).

This identity turns out to be quite useful in this scenario due to the following key observation, which stems from the rotation- like nature of the $U_j(\theta_j)$ gate:

$$K(\theta_j) = U_j(\theta_j)^\dagger K U_j(\theta_j) = A + \cos(\theta_j)B + \sin(\theta_j)C,$$

where K is an arbitrary operator and A, B and C are operators independent of θ_j and involve only

K and H_j . The parameter- shift rule for the operator K can easily be deduced now,

$$\frac{\partial K(\theta_j)}{\partial \theta_j} = \frac{K(\theta_j + h) - K(\theta_j - h)}{2 \sin(h)}.$$

Note that, even if the preceding expression looks like a finite-difference approximation, it is actually exact and we can use it to analytically estimate the gradient of expectation values whenever the previously discussed rotation- like property holds. Thus, we further get:

$$g_j(\theta) = \frac{f(\theta + h\mathbf{e}_j) - f(\theta - h\mathbf{e}_j)}{2 \sin(h)},$$

where $\mathbf{e}_j$ is the unit vector along the θ_j axis.

Putting $h = \pi/2$ in the previous equation yields the famous result:

$$g_j(\theta) = \frac{f(\theta + \frac{\pi}{2}\mathbf{e}_j) - f(\theta - \frac{\pi}{2}\mathbf{e}_j)}{2}.$$

Second- order derivative

Applying the rule for the first- order derivative twice gives the formula for the second- order derivative as :

$$g_{j_1, j_2}(\theta) = \frac{f(\theta + h_1\mathbf{e}_{j_1} + h_2\mathbf{e}_{j_2}) - f(\theta - h_1\mathbf{e}_{j_1} + h_2\mathbf{e}_{j_2}) - f(\theta + h_1\mathbf{e}_{j_1} - h_2\mathbf{e}_{j_2}) + f(\theta - h_1\mathbf{e}_{j_1} - h_2\mathbf{e}_{j_2})}{4 \sin(h_1) \sin(h_2)}$$

which, for $h_1 = h_2 = h$ and $j_1 = j_2 = j$ yields:

$$g_{j,j}(\theta) = \frac{f(\theta + 2h\mathbf{e}_j) - 2f(\theta) + f(\theta - 2h\mathbf{e}_j)}{(2 \sin(h))^2}.$$

Again, $\pi/2$ serves as a good choice for h which further simplifies the expression to:

$$g_{j,j}(\theta) = \frac{f(\theta + \pi\mathbf{e}_j) - f(\theta)}{2},$$

since $f(\theta + \pi\mathbf{e}_j) = f(\theta - \pi\mathbf{e}_j)$ due to the *trigonometric* nature of f discussed in Section 10.1.

10.2 Central Difference Approximation

In situations where an exact gradient rule like the parameter-shift is not applicable, such as when the generator G_j of the gate $U_j(\theta_j)$ does not square to identity, the gradient can still be approximated numerically. One such approach is the *central difference approximation*, which estimates the derivative of a function $f(\boldsymbol{\theta})$ by evaluating it at symmetric points around θ_j :

$$\frac{\partial f}{\partial \theta_j} \approx \frac{f(\boldsymbol{\theta} + h\mathbf{e}_j) - f(\boldsymbol{\theta} - h\mathbf{e}_j)}{2h}$$

Unlike the previous formulae, here, h is taken to be a small shift parameter, typically chosen to be something like 10^{-3} . The method is “central” because it samples both forward and backward values

centered around θ_j , giving a more accurate estimate of the slope than one-sided methods. This approximation can also be found using the small-angle approximation $\sin(h) \approx h$ in the parameter-shift rule formula as $h \rightarrow 0$.

Although not exact, this method works generically for arbitrary parameterized unitaries and does not depend on any special structure of the circuit. However, its accuracy depends on the choice of h and can be affected by shot noise and hardware errors when implemented on real quantum devices.

10.3 Application to Variational Quantum Eigensolver

Gradient estimation techniques find direct application in the *variational quantum eigensolver* (VQE), a hybrid algorithm designed to approximate the ground state energy of a quantum system. The core idea of VQE relies on the variational principle, which guarantees that for any normalized trial state $|\psi(\boldsymbol{\theta})\rangle$ prepared by a parameterized circuit $U(\boldsymbol{\theta})$, the expectation value

$$E(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle$$

satisfies $E(\boldsymbol{\theta}) \geq E_0$, where E_0 is the true ground state energy of the Hamiltonian H . Therefore, minimizing $E(\boldsymbol{\theta})$ over the parameter space effectively brings the system closer to the ground state.

The Hamiltonian H is typically expressed as a linear combination of tensor products of Pauli operators:

$$H = \sum_{\alpha} c_{\alpha} P_{\alpha},$$

where each P_{α} is a Pauli string (e.g., $I \otimes X \otimes Z$) and $c_{\alpha} \in \mathbb{R}$. Consequently, the energy expectation decomposes as

$$E(\boldsymbol{\theta}) = \sum_{\alpha} c_{\alpha} \langle \psi(\boldsymbol{\theta}) | P_{\alpha} | \psi(\boldsymbol{\theta}) \rangle,$$

enabling term-wise evaluation of expectations on quantum hardware.

Since VQE is an optimization problem over continuous parameters $\boldsymbol{\theta}$, gradient-based optimization is often preferred for its faster convergence, especially in high-dimensional landscapes. The gradient $\nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta})$ is computed by taking derivatives of each expectation term, and the techniques introduced earlier, parameter-shift rule, central difference approximation, etc., are applied individually to each P_{α} .

Gradient-conscious implementations of VQE, such as those incorporating analytic gradients or hybrid estimators, are particularly useful on noisy intermediate-scale quantum (NISQ) machines. They enhance precision as well as efficiency and are the foundation for complex quantum chemistry simulations and materials modeling.